

Relatório – Qualidade e Teste de Software

Tema: Aplicação prática da ISO/IEC 25010 com teste unitário – Carrinho de Compras

Disciplina: Qualidade e Teste de Software – Prof. Hugo Menezes Barra

Aluno: Arthur Moreira Santos

Repositório: https://github.com/naivepage13/Trabalho01_Mercado

1. Análise com a ISO/IEC 25010

1.1 Adequação Funcional

A adequação funcional mede se o software entrega as funções corretas com a exatidão esperada. Os testes verificam cada regra de negócio do checkout com valores calculados à mão: desconto de 10% a partir de R\$ 100,00, de 20% a partir de R\$ 500,00 e frete grátis a partir de R\$ 200,00 (taxa de R\$ 20,00 abaixo disso). Além dos casos normais (R\$ 50, R\$ 150, R\$ 300, R\$ 1000), a análise de valor limite exercita os pontos de transição (99,99/100,00, 499,99/500,00 e 199,99/200,00), onde erros de operador como `>` no lugar de `>=` aparecem. Assim, a suíte garante completude (todas as regras são cobertas) e correção (resultados exatos, comparados com `pytest.approx` para tolerar a aritmética de ponto flutuante).

1.2 Confiabilidade

A confiabilidade trata da capacidade de operar de forma estável, inclusive diante de entradas inválidas. O componente rejeita preços nulos ou negativos e impede o checkout de carrinhos vazios, lançando `ValueError` com mensagem clara. Os testes confirmam que a exceção é de fato lançada (`pytest.raises`) para 0, -0,01 e -10,00 e para o carrinho vazio, e que um item rejeitado não é inserido no carrinho (estado permanece consistente). Dessa forma, dados incorretos não geram totais absurdos nem corrompem o estado, e falhas são previsíveis e tratáveis pelo chamador (tolerância a falhas e recuperabilidade).

1.3 Manutenibilidade

A manutenibilidade envolve a facilidade de modificar o código sem introduzir defeitos. Uma suíte automatizada funciona como rede de segurança: após refatorar (por exemplo, trocar os `if/elif` por uma tabela de faixas, ou migrar de `float` para `Decimal`), basta executar `pytest` em menos de um segundo para saber se alguma regra existente foi quebrada. Os testes também documentam o comportamento esperado, têm nomes descritivos e são independentes entre si (analísabilidade e testabilidade), reduzindo o custo e o risco de evoluções futuras, como novas faixas de desconto.

2. Mapeamento dos cenários de teste

Categoria	Cenário	Entrada	Resultado esperado
Fluxo principal	Sem desconto, frete padrão	R\$ 50,00	Desc. 0; frete 20; total 70,00
Fluxo principal	Soma de itens	10 + 20,50 + 30	Produtos 60,50; total 80,50
Fluxo principal	10% com frete pago	R\$ 150,00	Desc. 15; frete 20; total 155,00
Fluxo principal	10% com frete grátis	R\$ 300,00	Desc. 30; frete 0; total 270,00
Fluxo principal	20% com frete grátis	R\$ 1000,00	Desc. 200; frete 0; total 800,00
Limite desconto	Abaixo de 100	R\$ 99,99	Desc. 0; total 119,99
Limite desconto	Exatamente 100	R\$ 100,00	Desc. 10; total 110,00
Limite desconto	Abaixo de 500	R\$ 499,99	Desc. 10% (49,999); total 449,991
Limite desconto	Exatamente 500	R\$ 500,00	Desc. 20% (100); total 400,00
Limite frete	Abaixo de 200	R\$ 199,99	Frete 20
Limite frete	Exatamente 200	R\$ 200,00	Frete 0; total 180,00
Exceção	Preço zero / negativo	0; -0,01; -10	<code>ValueError</code> ; carrinho segue vazio
Exceção	Checkout vazio	carrinho vazio	<code>ValueError</code>
Extra	Carrinho vazio: totais	carrinho vazio	Produtos 0; desc. 0; frete 0

3. Execução dos testes

Saída do terminal com pytest -v: 22 testes executados, 22 aprovados.

```
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0 --
C:\Users\arthur\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\arthur\Downloads\mercado
plugins: cov-7.1.0
collecting ... collected 22 items
test_carrinho.py::test_compra_sem_desconto_com_frete_padrao PASSED [ 4%]
test_carrinho.py::test_soma_de_varios_itens PASSED [ 9%]
test_carrinho.py::test_desconto_10_por_cento_com_frete_pago PASSED [ 13%]
test_carrinho.py::test_desconto_10_por_cento_com_frete_gratis PASSED [ 18%]
test_carrinho.py::test_desconto_20_por_cento_com_frete_gratis PASSED [ 22%]
test_carrinho.py::test_limite_faixa_de_desconto[99.99-0.0] PASSED [ 27%]
test_carrinho.py::test_limite_faixa_de_desconto[100.0-10.0] PASSED [ 31%]
test_carrinho.py::test_limite_faixa_de_desconto[499.99-49.999] PASSED [ 36%]
test_carrinho.py::test_limite_faixa_de_desconto[500.0-100.0] PASSED [ 40%]
test_carrinho.py::test_total_final_em_99_99 PASSED [ 45%]
test_carrinho.py::test_total_final_em_100_00 PASSED [ 50%]
test_carrinho.py::test_total_final_em_499_99 PASSED [ 54%]
test_carrinho.py::test_total_final_em_500_00 PASSED [ 59%]
test_carrinho.py::test_limite_frete_gratis[199.99-20.0] PASSED [ 63%]
test_carrinho.py::test_limite_frete_gratis[200.0-0.0] PASSED [ 68%]
test_carrinho.py::test_total_final_em_199_99 PASSED [ 72%]
test_carrinho.py::test_total_final_em_200_00 PASSED [ 77%]
test_carrinho.py::test_adicionar_item_com_preco_invalido_lanca_value_error[0] PASSED [ 81%]
test_carrinho.py::test_adicionar_item_com_preco_invalido_lanca_value_error[-0.01] PASSED [ 86%]
test_carrinho.py::test_adicionar_item_com_preco_invalido_lanca_value_error[-10.0] PASSED [ 90%]
test_carrinho.py::test_calcular_total_final_em_carrinho_vazio_lanca_value_error PASSED [ 95%]
test_carrinho.py::test_carrinho_vazio_tem_total_desconto_e_frete_zerados PASSED [100%]
===== 22 passed in 0.03s =====
```

4. Considerações finais

A suíte cobre os três pilares analisados e todos os pontos de transição das regras de negócio. Observação: o cálculo usa float, o que pode gerar pequenas imprecisões em valores monetários (por isso a comparação com tolerância); em produção, recomenda-se Decimal, e os testes existentes permitiriam essa migração com segurança. Também não há validação de tipo para o preço (ex.: texto), o que poderia ser um próximo caso de teste.